

Mini Banking System

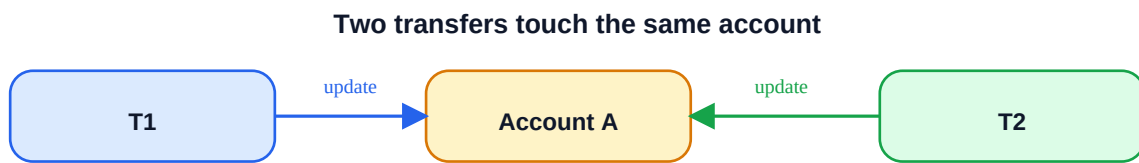
Transaction Flow and Distribution Planning

Computer Systems and Their Fundamentals - Database Management Systems

Student	Taher Amine ELHOUARI
Part 1	Transaction management, concurrency control, locking and safe scheduling
Part 2	Horizontal/vertical fragmentation, replication and allocation planning

Goal: preserve financial correctness when concurrent transactions touch the same account, while designing a branch-aware distributed database for Tunis, Sousse, and Sfax.

Part 1 - Transaction Management



Without coordination, both may read the same old balance and overwrite each other.

1. Concurrency issue

The main risk is a **lost update**. Assume Account A has a balance of 1,000. Transaction T1 transfers 100 from A, while T2 simultaneously transfers 50 from A. If both transactions read the original 1,000 before either writes, T1 may write 900 and T2 may later write 950. The 100 debit from T1 has effectively disappeared. This violates transaction isolation and produces an incorrect balance.

A **dirty read** is another possible problem if one transaction reads a balance written by another transaction that has not yet committed. If the first transaction later rolls back, the second transaction has acted on a value that never became permanent.

2. Locking mechanism

Use **shared (S) locks** for read-only balance queries and **exclusive (X) locks** for balance updates. A money transfer should acquire an exclusive lock on every account whose balance it changes, keep those locks until commit/rollback, and then release them.

To reduce deadlock risk when a transfer touches two accounts, always acquire account locks in a deterministic order (for example, lower `account_id` first).

3. Pessimistic vs optimistic locking

I would use **pessimistic locking** for the transfer path. Banking balances are high-integrity data, and conflicts involving the same account are realistic. It is safer to prevent concurrent writers from entering the critical section than to allow them to proceed and detect conflicts later. Optimistic locking can still be suitable for lower-contention or read-heavy data, but the balance-debit/credit operation should favor correctness and predictable isolation.

4. Safe concurrent schedule

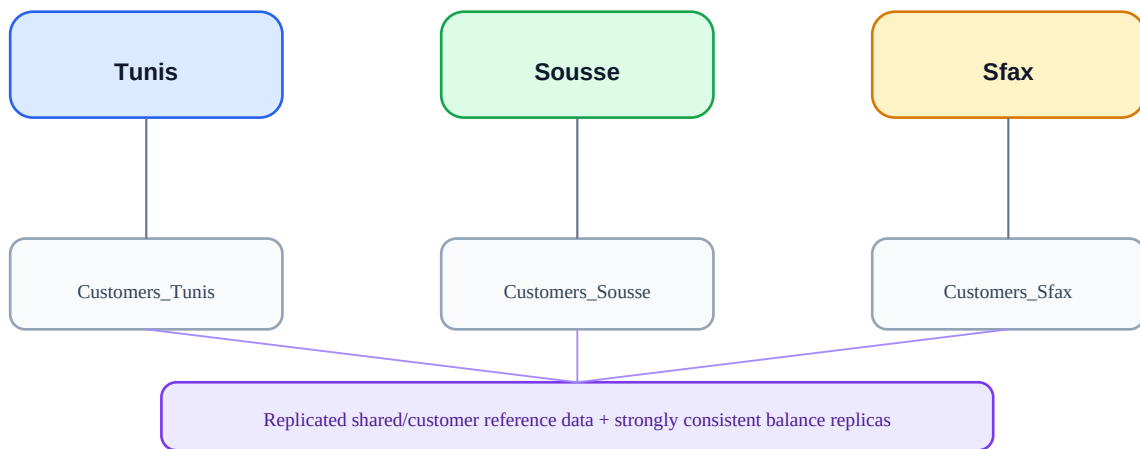
Step	Transaction T1	Transaction T2	Account A	Safe?
1	Request X-lock(A)	-	1,000	Yes
2	Lock granted; read A=1,000	Request X-lock(A)	1,000	Yes - T2 waits
3	Write A=900	WAIT	900 (uncommitted)	Yes
4	COMMIT; unlock(A)	WAIT	900	Yes
5	-	X-lock(A) granted; read A=900	900	Yes
6	-	Write A=850	850 (uncommitted)	Yes

Step	Transaction T1	Transaction T2	Account A	Safe?
7	-	COMMIT; unlock(A)	850	Yes

Conclusion: The schedule is safe and serializable. T2 cannot read or overwrite Account A while T1 holds the exclusive lock, so the final balance correctly reflects both transfers: $1,000 - 100 - 50 = 850$.

Part 2 - Distributed Database Planning

High-Level Distributed Database Plan



1. Horizontal fragmentation of Customers

Horizontally fragment the **Customers** table by the branch that owns or primarily serves the customer. Each branch stores the rows it uses most frequently, reducing unnecessary remote reads.

Fragment	Selection rule	Primary location
Customers_Tunis	branch = 'Tunis'	Tunis
Customers_Sousse	branch = 'Sousse'	Sousse
Customers_Sfax	branch = 'Sfax'	Sfax

Conceptually: **Customers** = **Customers_Tunis UNION Customers_Sousse UNION Customers_Sfax**. The fragments should be disjoint by branch_id so a customer row is not accidentally mastered by two branches.

2. Vertical separation

Separate authentication/contact-sensitive data into a dedicated table such as **CustomerCredentials**. For example:

Table	Example columns
Customers	customer_id, name, branch_id, address, date_of_birth
CustomerCredentials	customer_id, email, password_hash, auth_provider, mfa_enabled

This reduces exposure of login-related fields, supports tighter access controls, and avoids repeatedly retrieving sensitive authentication data during ordinary customer-service queries.

3. Replication strategy

Data	Replicate to all branches?	Recommendation / reason
Customer reference information	Yes	Branch staff may need to identify customers from any branch. Replication improves availability and lookup speed.

Data	Replicate to all branches?	Recommendation / reason
Account balances	Yes, but with strong consistency	Balances are operationally critical. Replicas can support availability, but writes must be coordinated so there is one authoritative value and no divergent balance copies.
Full transaction history	No, not fully	History grows quickly. Keep detailed records close to the branch/partition that owns them, then replicate backups, summaries, or recent data as required.

Important banking design principle: replication must not create independently writable balance copies. The system needs a single authoritative ordering for balance-changing transactions (for example, a primary/leader or consensus-backed write path).

4. Static or dynamic allocation for transaction history?

I would choose **dynamic allocation** for transaction history. Transaction volume is not guaranteed to be equal in Tunis, Sousse, and Sfax. Dynamic placement allows the system to move, replicate, or archive history according to workload, storage pressure, access patterns, and operational demand.

For example, recent Tunis transactions can stay close to Tunis for fast access while older records can be moved to lower-cost archival storage or replicated to a central analytics location. The trade-off is higher management complexity, but the flexibility is valuable for a growing transaction dataset.

Integrated Design Summary

Area	Decision
Primary concurrency risk	Lost update; dirty reads are also possible without isolation.
Read locking	Shared lock for balance reads when strict consistency is required.
Write locking	Exclusive lock held until transaction commit/rollback.
Locking style	Pessimistic locking for balance-changing transfers.
Deadlock mitigation	Acquire account locks in a deterministic account_id order.
Customer fragmentation	Horizontal fragments by branch: Tunis, Sousse, Sfax.
Vertical separation	Move email/login/authentication data to CustomerCredentials.
Replicated data	Customer reference data and strongly consistent balance replicas.
Transaction history	Branch-local/partitioned detail; dynamic allocation and selective replication/archival.

The proposed design prioritizes correctness for financial transactions and locality/availability for distributed branch operations. Concurrency control prevents conflicting balance updates, while fragmentation and controlled replication reduce unnecessary remote access without sacrificing consistency.

Prepared for the Mini Banking System - Transaction Flow and Distribution Planning checkpoint.